

Chapter 2

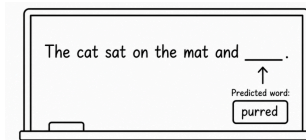
Pretraining, Fine-tuning and Prompting

Module 1: Foundations of Large Language Models

In the last chapter we covered what LLMs are. Now we're going to dig in to how they actually get built — and how you interact with them effectively. Three stages: pretraining, fine-tuning, and prompting. Let's go!

Pretraining

The foundation

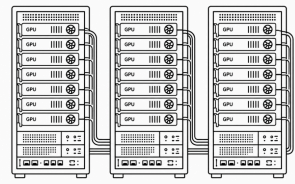


- Self-supervised learning on unlabeled text data
- Predict next words/tokens to learn grammar, facts, and patterns
- No human labels needed—model learns from context alone
- Example: Predicting "The sky is ____" → "blue"

Pretraining is where it all begins. The model reads a massive amount of text and learns by predicting what comes next. Given "The sky is ____", it learns to guess "blue." No human labels, no hand-holding — just a massive dataset and one simple objective repeated trillions of times. By the end, it has a surprisingly broad understanding of language, facts, and structure.

Pretraining Mechanics

Billions of parameters in action



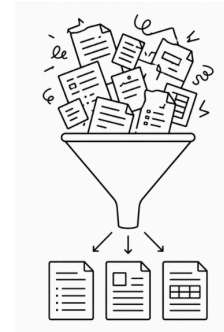
- Uses Transformer architecture:
- Objective: Masked Language Modeling (MLM) or Next Token Prediction (NTP)
- Trained on GPUs/TPUs for weeks/months with massive compute
- Scales with data and parameters: Bigger = Better (often)

Under the hood, pretraining runs on the Transformer architecture — the same basic design powering most modern LLMs. Training happens on clusters of GPUs or TPUs for weeks, sometimes months, consuming a staggering amount of compute. The key insight is that scale matters: more parameters, more data, better results. Usually.

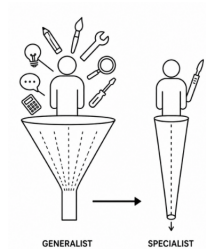
Pretraining Datasets and Challenges

Quality over quantity

- Common datasets: Common Crawl, Wikipedia, books, code - all contained in the Pile
- Challenges: Data quality, biases, copyright issues
- Deduplication and filtering: Remove noise for better learning
- Outcome: A 'world model' of language, but not perfect



The data comes from everywhere — web scrapes, Wikipedia, books, code. But raw internet data is messy, biased, and raises copyright questions. Teams filter aggressively to get to something usable. The model ends up with a broad world model built from text — but it inherits whatever was in that data, good and bad.



- Supervised adaptation of pre-trained model on labeled task data
- Goal: Improve performance for specific domains
- Smaller datasets than pretraining
- Keeps most pre-trained weights, updates a subset

Introduction to Fine-Tuning

Customization for real-world use

Where pretraining gives you a generalist, Fine-tuning turns that model into a specialist. You take that pretrained model and train it further on a smaller, targeted dataset — customer support, medical Q&A, legal documents. Most of the original weights stay frozen; you're just nudging it in the right direction. Much faster and cheaper than pretraining.

Supervised Fine-tuning Techniques

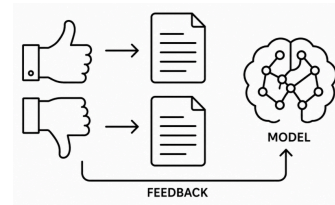
General to task-specific

- Supervised Fine-tuning (SFT): Train on input-output pairs "Translate: Hello" → "Bonjour"
- Instruction Tuning: Follow instructions format "Write a poem about..." → Poem
- Benefits: Better alignment with user intent, reduced errors
- Tools: Hugging Face Transformers library for easy implementation

Two main approaches. Supervised Fine-tuning uses direct input-output pairs — "Translate: Hello" gets you "Bonjour." Instruction Tuning trains on prompts written as commands, helping the model understand what you actually want rather than what it statistically expects. Tools like Hugging Face make this surprisingly accessible.

Reinforcement from Human Feedback

Human-centric refinement



- Fine-tune using human preferences
- SCollect rankings → Train reward model → Optimize with PPO
- ChatGPT uses this to make outputs safer and more useful
- Alternatives include DPO (Direct Preference Optimization) for efficiency

RLHF is how models learn to be not just capable, but actually helpful. Human raters compare outputs and rank them. Those rankings train a reward model, which guides the LLM toward better behavior. This is what turned early GPT into something people actually wanted to use. A newer approach called DPO is becoming popular as a simpler alternative.

The Power of Prompting

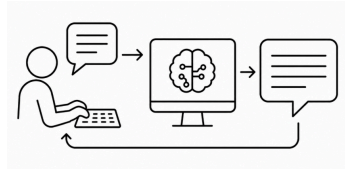
Hacking the model's mind

- Zero/few-shot interaction:
No retraining needed—just craft the input
- Controls model behavior without changing weights
- Emergent abilities enable complex tasks from simple prompts in large models
- "Explain quantum physics like I'm 5"

Once a model is trained, you don't need to touch the weights to change its behavior — you just change what you ask it. Zero-shot, few-shot, complex multi-step tasks — all from a well-crafted input. This is what we're going to spend a lot of time on in this course. Once we've covered the basics of LLMs, we're going to explore prompt engineering, with some hands-on exercises to help you master the topic.

Prompt Engineering Techniques

Trial and error pays off



- Chain-of-Thought (CoT):
"Think step by step" for reasoning tasks
- Role-Playing: "You are a helpful teacher..." to set persona
- Few-Shot: Provide 1-3 examples before the query
- Iterative Refinement: Start broad, then specify

A few techniques worth knowing. Chain-of-Thought — just adding "think step by step" — meaningfully improves accuracy on reasoning tasks. Role prompting sets a persona. Few-shot gives examples to pattern-match against. And always iterate — your first prompt is rarely your best one.

Key Takeaways and Next Steps

End of chapter - quiz time!

- Pretraining: Scale data and compute for broad knowledge
- Fine-tuning:
 - Use targeted data for specialization
 - Consider RLHF for alignment
- Prompting: Be clear, specific, and iterative
- Experiment ethically: Watch for biases in all stages
- Next: Dive into Tokenization, Embeddings, and Attention

Pretraining builds the foundation. Fine-tuning specializes it. Prompting is how you steer it day to day. One thing worth keeping in mind at every stage — biases can creep in from the data, the raters, the prompts themselves. Next up: Tokenization, Embeddings, and Attention Mechanisms.